

RESUMEN DISEÑO Y PROGRAMACIÓN ORIENTADA A OBJETOS

Diseño y Programación Orientada a Objetos (POO)	1
Definiciones Técnicas Literales	1
Elementos y Conceptos Principales del Modelo	2
Componentes Software: Clases y Objetos	3
Definición de Clase y Objeto	3
Visibilidad y Modificadores de Acceso (+, -, #, ~)	3
Clases Abstractas vs. Interfaces: Definiciones y Diferencias	4
Herencia y Especialización	4
Métodos: Sobrecarga (Overloading) vs. Sobrescritura (Overriding)	5
Patrones de Diseño (GoF - Gang of Four)	5
Clasificación Integral de Patrones GoF	5
Definiciones de Patrones Específicos de Examen	6
Patrones de Arquitectura: El modelo MVC (Modelo-Vista-Controlador)	7
Lenguaje de Modelado Unificado (UML)	8
Tipos de Diagramas en UML 2.x (ultima versión 2.5)	8
Relaciones en UML	9

Diseño y Programación Orientada a Objetos (POO)

Definiciones Técnicas Literales

- **Programación Orientada a Objetos (POO):** "Paradigma de programación que usa objetos y sus interacciones para diseñar aplicaciones y programas informáticos; es una forma especial de programar, más cercana a como expresaríamos las cosas en la vida real, en términos de objetos, propiedades y métodos".
- **Diseño Orientado a Objetos:** "Fase inicial de la metodología orientada a objetos para el desarrollo de software. Es la disciplina que define los objetos y sus interacciones para

resolver un problema de negocio que fue identificado y documentado durante el análisis".

- **SNAP (Strategic Networked Applications Platform):** Plataforma de la que surge la programación orientada a objetos.

Elementos y Conceptos Principales del Modelo

- **Abstracción:** "Se centra en la visión externa de un objeto y sirve para separar el comportamiento esencial de un objeto de su implementación concreta".
- **Encapsulación (u Ocultamiento):** "Mecanismo que permite juntar el código y los datos y que mantiene a ambos alejados de posibles usos indebidos. El acceso al código y a los datos se realiza de forma controlada a través de una interfaz definida".
- **Jerarquía:** "Clasificación u ordenación de abstracciones. Las dos jerarquías más importantes en un sistema complejo son su estructura de clases y su estructura de objetos".
- **Modularidad:** "Propiedad que posee un sistema que ha sido descompuesto en un conjunto de módulos cohesivos y débilmente acoplados".
- **Concurrencia:** "Propiedad que distingue un objeto activo de uno no activo. Permite que diferentes objetos actúen al mismo tiempo, usando distintos hilos (threads) de control".
- **Persistencia:** "Propiedad por la cual la existencia de un objeto trasciende en el tiempo (sigue existiendo después de que su creador deje de usarlo) o en el espacio".
- **Tipificado:** "Imposición de una clase a un objeto de tal modo que objetos de diferentes tipos no se puedan intercambiar, o se puedan intercambiar sólo de forma restringida".

Enfoque CPS: El tribunal suele preguntar por las características que NO corresponden al modelo orientado a objetos. Un distractor común es incluir la durabilidad (que es de ACID en bases de datos) como característica de la POO.

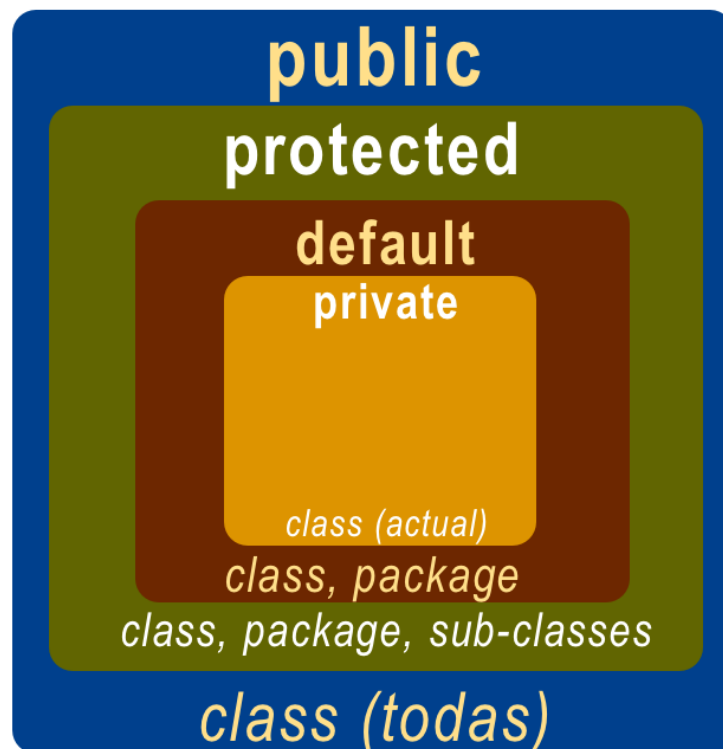
Componentes Software: Clases y Objetos

Definición de Clase y Objeto

- **Clase:** "Modelo que se usa para construir objetos similares. Es un tipo de dato definido por el usuario que determina las estructuras de datos que lo forman y las funciones asociadas con él".
- **Objeto:** "Conjunto complejo de datos y programas que poseen estructura y forman parte de una organización". "Abstracción de un dato y su comportamiento".
- **Instancia:** "Una realización específica de una clase".

Visibilidad y Modificadores de Acceso (+, -, #, ~)

- **Público (+):** Acceso desde cualquier clase.
- **Privado (-):** Acceso solo desde la propia clase.
- **Protegido (#):** Acceso desde la propia clase y sus subclases (herencia).
- **Paquete/Package (~):** Acceso solo desde clases del mismo paquete.



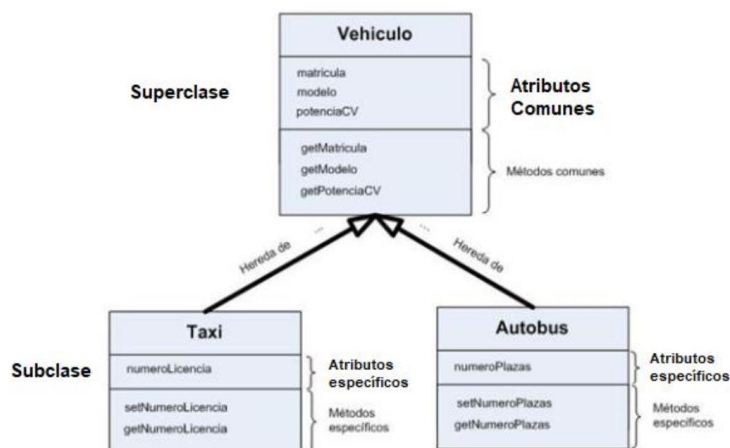
Clases Abstractas vs. Interfaces: Definiciones y Diferencias

- **Clase Abstracta:** Puede tener métodos implementados y métodos sin implementar (abstractos). No se pueden instanciar. Representa un "es un".
- **Interface:** En su concepto puro, solo define firmas de métodos (qué hacer, no cómo). Una clase "implementa" una interfaz. Representa un "es capaz de".

Enfoque CPS: Suelen preguntar si una interfaz puede tener atributos (normalmente solo constantes) o si se pueden instanciar (la respuesta es NO).

Herencia y Especialización

- **Herencia:** "Mecanismo que permite a una clase de objetos incorporar atributos y métodos de otra clase, añadiéndolos a los que ya posee. Permite definir una nueva clase en base a otra ya existente".



- **Generalización:** "Implica que la subclase hereda las propiedades, el comportamiento y las relaciones de la superclase, a la vez que puede añadir sus propias propiedades".
- **Superclase:** "La clase de la cual se hereda".
- **Subclase:** "La clase que hereda".
- **Herencia Múltiple:** "Una clase puede heredar de varias superclases". No todos los lenguajes la permiten (C++ sí, Java no).

Métodos: Sobrecarga (Overloading) vs. Sobrescritura (Overriding)

- **Métodos:** "Programa procedural escrito en cualquier lenguaje, que está asociado a un objeto determinado y cuya ejecución sólo puede desencadenarse a través de un mensaje recibido por éste o por sus descendientes".
- **Mensajes:** "Solicitudes que invocan operaciones específicas, con uno o más objetos como parámetros".
- **Polimorfismo:** "Posibilidad de asumir varias formas. Permite que una misma operación pueda llevarse a cabo de varias formas, en clases diferentes".
- **Sobrecarga (Overloading):** Mismo nombre de método, pero distintos parámetros (en la misma clase).
- **Sobrescritura (Overriding):** Una subclase redefine un método de la superclase con la misma firma (mismo nombre y mismos parámetros) para cambiar su comportamiento.

Enfoque CPS: Es vital distinguir entre polimorfismo y sobrecarga. CPS suele preguntar por la definición de sobrecarga como la capacidad de tener métodos con el mismo nombre pero distintos parámetros.

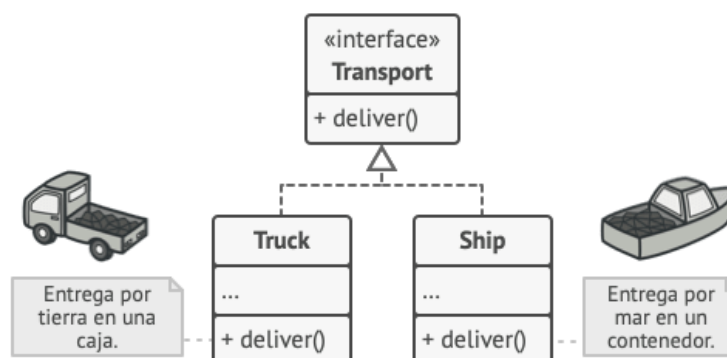
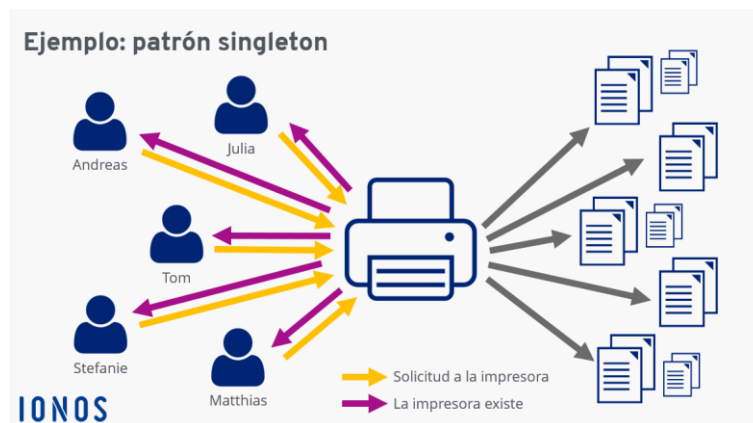
Patrones de Diseño (GoF - Gang of Four)

Los patrones de diseño son "soluciones para problemas típicos y recurrentes en el desarrollo de software".

Clasificación Integral de Patrones GoF

CATEGORÍA	ASOCIADO S A CLASES	ASOCIADO S A OBJETOS	DEFINICIÓN CLAVE
Creación	Abstract Factory, Builder	Factory Method, Prototype, Singleton	Conciernen al proceso de creación de objetos y toma de decisiones dinámicas.
Estructura	Class Adapter, Herencia múltiple	Object Adapter, Bridge, Composite, Decorator,	Tratan de la composición de clases y objetos para formar

		Facade, Flyweight, Proxy	estructuras complejas.
Comportamiento	Interpreter, Template Method	Chain of Responsibility, Command, Iterator, Mediator, Memento, Observer, State, Strategy, Visitor	Caracterizan las maneras en las que las clases u objetos interactúan y se distribuyen responsabilidades.



Definiciones de Patrones Específicos de Examen

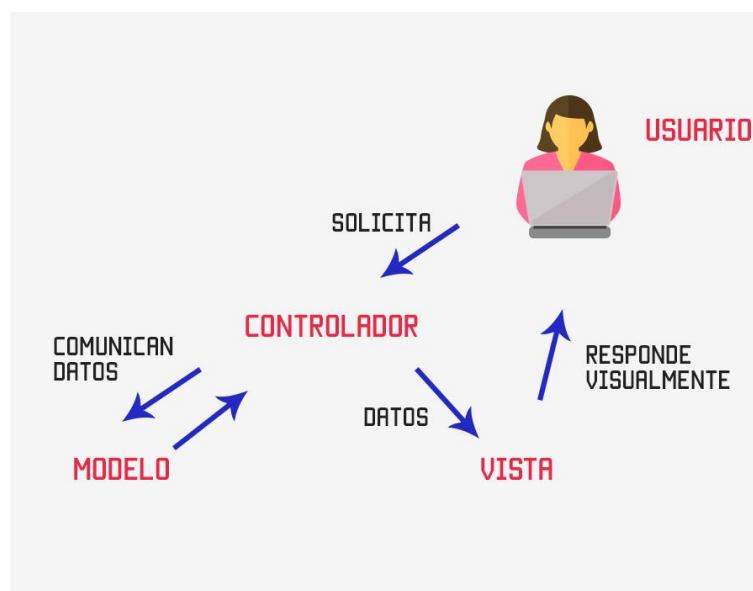
- **Singleton:** "Garantiza la existencia de una única instancia para una clase y la creación de un mecanismo de acceso global a dicha instancia".

- **Adapter:** "Permite que clases con interfaces incompatibles trabajen juntas convirtiendo la interfaz de una clase en otra interfaz esperada".
- **Composite:** "Compone objetos en estructuras de árbol para representar jerarquías parte-todo. Permite tratar objetos individuales y composiciones de manera uniforme".
- **Memento:** "Permite capturar y restaurar el estado interno de un objeto sin violar la encapsulación".
- **GRASP (General Responsibility Assignment Software Patterns):** Patrones generales para asignación de responsabilidades.

Enfoque CPS: El examen de TAI 2024 incluyó una pregunta trampa donde se asignaba la definición del patrón Decorator al patrón Composite. Memorizar que Composite = Estructura de árbol.

Patrones de Arquitectura: El modelo MVC (Modelo-Vista-Controlador)

- **Modelo:** Gestión de datos y lógica de negocio.
- **Vista:** Interfaz de usuario (presentación).
- **Controlador:** Intermediario que gestiona los eventos y comunica el modelo con la vista.



Lenguaje de Modelado Unificado (UML)

UML es un "lenguaje gráfico para visualizar, especificar, construir y documentar un sistema de software. Ofrece un estándar para describir un 'plano' del sistema (modelo)".

Tipos de Diagramas en UML 2.x (última versión 2.5)

1. Diagramas Estructurales (Estáticos):

- **Diagramas de Clases:** "Muestra las clases de un sistema, atributos y operaciones de cada clase y la relación entre ellas". Una clase se representa con un rectángulo dividido en tres áreas (nombre, atributos, operaciones).
- **Diagramas de Objetos:** "Presenta los objetos y sus relaciones en un punto del tiempo".
- **Diagramas de Paquetes:** "Muestra cómo un sistema está dividido en agrupaciones lógicas y las dependencias entre esas agrupaciones".
- **Diagrama de Componentes:** Describe la organización y las dependencias entre componentes de software.
- **Diagrama de Despliegue (Deployment):** Representa la configuración de los nodos de procesamiento en tiempo de ejecución y los componentes que viven en ellos (hardware + software). Es el único que se centra en el hardware/topología.

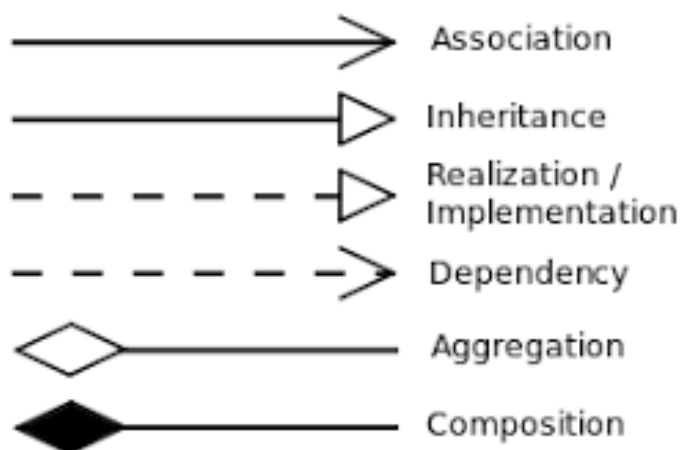
2. Diagramas de Comportamiento (Dinámicos):

- **Diagramas de Casos de Uso:** "Modelan la funcionalidad del sistema usando actores y casos de uso (servicios o funciones)".
- **Diagramas de Interacción:** "Modelos que describen cómo grupos de objetos colaboran para conseguir algún fin. Describe en detalle un determinado escenario de un caso de uso".
- **Diagrama de Secuencia:** "Muestra la mecánica de la interacción con base en tiempos". Representa acciones como "autenticación, firma, guardado en BD".
- **Diagrama de Comunicación (o Colaboración):** "Muestra cómo interactúan los objetos entre sí, con qué otros objetos tiene vínculos".

- **Diagramas de Estados (o Máquina de Estados):** Analiza los "estados, transiciones entre estados y los eventos que los provocan" para un objeto.
- **Diagramas de Actividades:** Enfatizan el proceso de comportamiento del sistema.

Relaciones en UML

- **Agregación:** "Relación jerárquica entre un objeto que representa la totalidad y las partes que lo componen". Se representa con un rombo hueco.
- **Composición:** Agregación fuerte donde la vida de la parte depende de la vida del todo. Se representa con un rombo lleno.
- **Dependencia:** "Relación semántica entre dos elementos, en la cual un cambio en un elemento puede afectar a la semántica de otro".



Enfoque CPS: Una pregunta recurrente es identificar qué diagrama NO pertenece a UML (ej. "Diagrama de actuación" o "Diagrama de historias de usuario" son distractores comunes). También es frecuente pedir el diagrama adecuado para representar un flujo temporal (Respuesta: Secuencia).

En UML, la herencia se representa con una línea sólida y un triángulo hueco apuntando a la superclase. Es muy común que en el examen pongan varias flechas (rellena, punteada, etc.) para que identifiques cuál es la de herencia.